Πthon Arena

American University of Beirut

EECE 350: Computer Networks

Dr. Sanaa Sharafeddine

Submission Date: April 26, 2026

Sari Sfeir
202510403
scs12@mail.aub.edu
Effort: 36%

Alwalid Baydoun
202503189
Amb90@mail.aub.edu
Effort: 37%

Georgio Sleiman
202504487
Ggs17@mail.aub.edu
Effort: 27%

1. Introduction

Πthon Arena is a real-time multiplayer snake battle game implemented using Python and Pygame. The system's architecture is client-server, where multiple clients connect to a centralized server to participate in the game session.

The objective of this project is to design and implement a networked application that supports real-time interaction between users, focusing on networking concepts like socket programming, real time data exchange, and network protocols.

2. System Architecture

The system is based on a client-server model, where a centralized server manages all game logic and communication. All components interact over a TCP connection, forming a structured and continuous communication system.

2.1. Client Architecture

The client component:

1. Establishes connection with the server
2. Sends user inputs (movement, chat, actions)
3. Receives game state updates
4. Renders the game using Pygame
5. Handles user interface events

The client is divided into two main subsystems:

I. The first system:

Where we take care of the rendering, the inputs and the display updates.

II. The second system:

Maintains TCP connection

Sends messages asynchronously

Continuously receives server updates

Decodes JSON messages

Pushes messages into a thread-safe queue

We decided to separate the client into these two subsystems so that both could run simultaneously, ensuring that the game runs smoothly without the network threading blocking the UI.

2.2. Server Architecture

The server is the core of the system, responsible for managing all clients and maintaining the authoritative game state.

The server:

1. Accept incoming connections
2. Manage multiple clients simultaneously
3. Validate usernames
4. Maintain lobby and player states
5. Run the game loop
6. Process player inputs
7. Broadcast updates to all clients

The server like the client is composed of two components:

1. Connection Handling Layer:
   Each client is handled by this dedicated object: “class ClientHandler” (line 17 of server.py) This object receives messages from a specific client and parses JSON messages, makes actions like JOIN, and INPUT etc. available and then sends responses

   Each client runs its own thread which enables multiple clients to interact simultaneously and prevents one slow client from blocking others. This demonstrates multi-threaded server design.

2. Game Layer:
   The game logic is implemented in: “class GameState” which maintains full game state: snake position, health values, obstacle, and pie locations. Processes movement inputs in addition to detecting collisions, update scores, and determine when the game ends.

The server runs a loop and at each iteration it processes the user inputs, updates the game state, and broadcasts the state to clients. This ensures synchronization and consistent state across all clients.

2.3. Communication architecture

Communication is based on persistent TCP socket connections between each client and the server.

For example: the user presses the up button while the game is already running, the button pressed is an input that will be sent to the server to process and the broadcast the updated game state.

The connection established is persistent and reliable requiring a handshake between the server and the client, tolerating no loss.

2.4. Protocol layer

The protocol layer defines how data is structured and interpreted, it serves as a shared interface between the client and server. It defines all message types, game constants, and communication rules used during data exchange.

Instead of hardcoding strings and values in multiple places, both the client and server import this file to ensure consistency and maintainability. So it defines what we are sending over the TCP connection we established in the client and server.

2.5. Data flow and Summary

The system works as follows:

1. Client connects via TCP using the socket
2. Client sends JOIN request (defined in protocol.py)
3. Server validates and responds
4. Clients send INPUT messages continuously
5. Server updates game state
6. Server broadcasts GAME_STATE at fixed intervals
7. Clients render received state

The perk of having such system is:

1. Centralized control (one server handling clients)
2. Scalability (multiple clients supported)
3. Responsiveness (multithreading prevents blocks)
4. Maintainability (the systems are separated so changing one part is easy for both maintenances, debugging, and future update/upgrades.

3. Project Features & Implementation Status

The project has achieved 100% completion of the required technical specifications, with multiple additional 'stretch goals' successfully implemented to enhance user experience.

Feature	Implementation Detail	Status
TCP Client-Server Communication	Uses Python sockets (socket.AF_INET, SOCK_STREAM) to establish persistent TCP connections between clients and server.	Implemented
Username Validation	Server checks uniqueness using a dictionary of active usernames before sending JOIN_OK or JOIN_ERR	Implemented
Multiplayer Support	Server handles multiple clients using threads (ClientHandler), each managing a separate connection.	Implemented
Real-time Spectating	Fans join matches mid-way through and watch state updates.	Implemented

Pie Mutations	Spawning of Hot Sauce, Whipped Cream, and Blueberry pies.	Implemented
Client-Server Protocol (JSON)	Messages are encoded using JSON and include a "type" field	Implemented
Real-time Game Updates	Server runs a game loop at fixed rate (10 ticks/sec) and broadcasts GAME_STATE messages to all clients.	Implemented
Chat System	Clients send CHAT messages; server forwards them using broadcast	Implemented
Customization	UI-based color selectors for player snakes.	Implemented
Music	Added in game music and the ability to mute it.	Implemented
Spectator Mode	Clients can send WATCH request and receive continuous game state without participating	Implemented
Cheer System	Spectators send CHEER messages; server broadcasts them to all connected clients.	Implemented
Tracking online users	Can see every online user and check if he is ready or not.	Implemented

4. Description of implemented functionalities

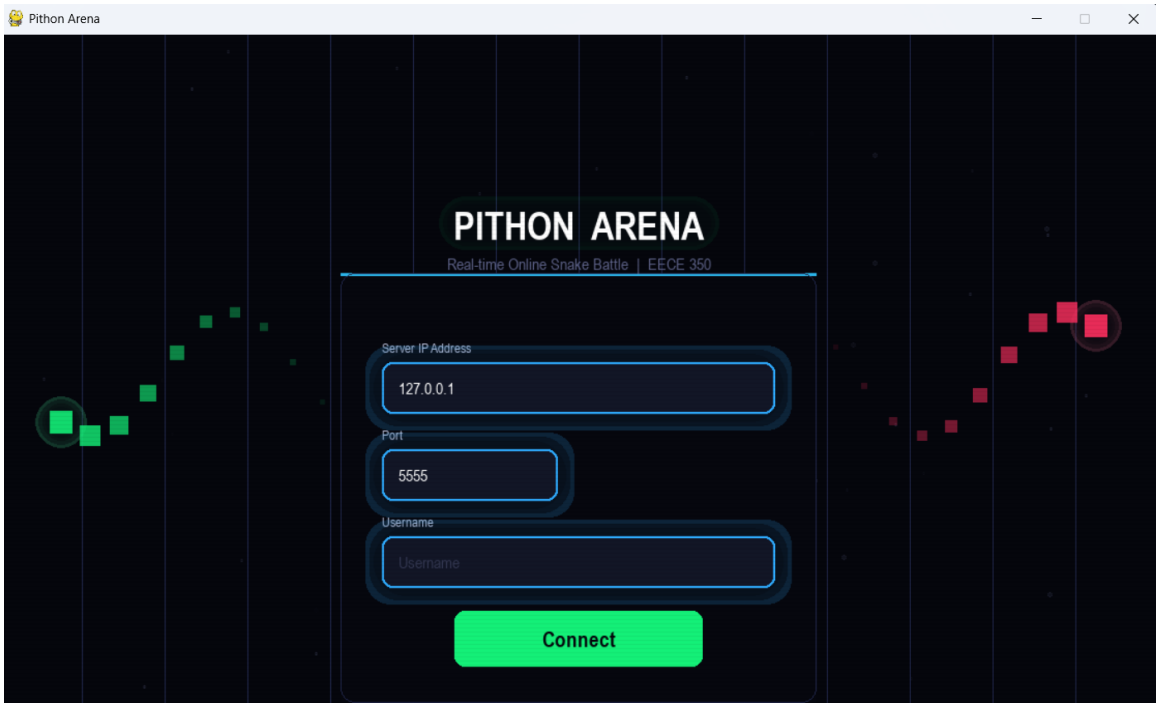
The system functionalities are implemented through a combination of client interactions and server processing. The client establishes a TCP connection with the server and sends structured JSON messages representing user actions such as movement, chat, and game requests. The server is responsible for handling all incoming messages, updating the game state, and broadcasting updates to connected clients.

Game state management is centralized at the server, where all game logic including movement validation, collision detection, and scoring is executed at the server level. Clients act as interfaces that render the received state without performing any computations.

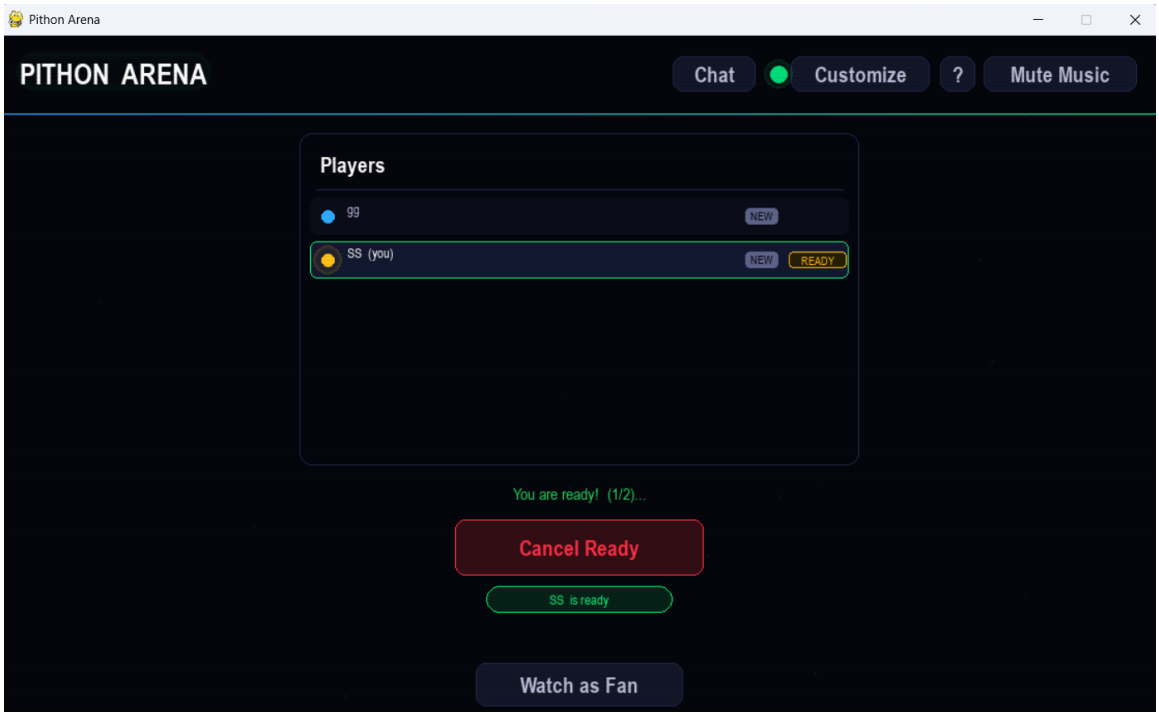
Real-time communication is achieved through a continuous game loop running at a fixed rate, where the server periodically sends updated game states to all clients. Multithreading is used both on the client and server to ensure seamless communication and responsiveness. The communication protocol is implemented using JSON messages defined in a shared protocol file, enabling structured and extensible interaction between system components.

Appendix A: Application Snapshots

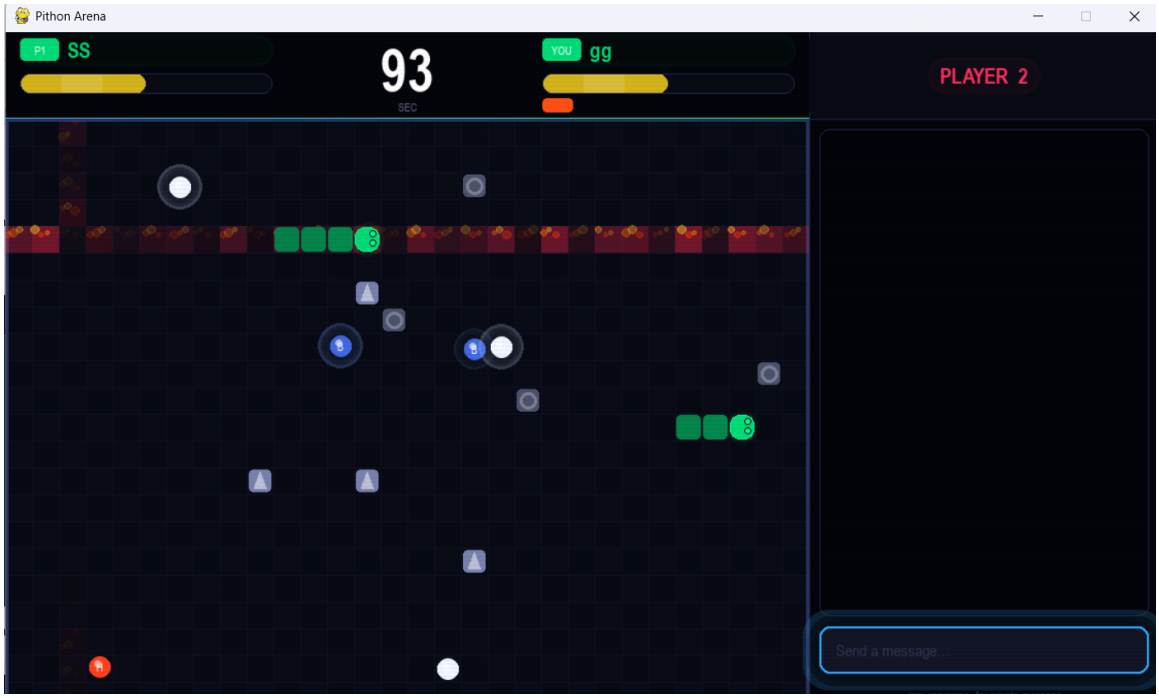
The following section presents the the Πthon Arena application:



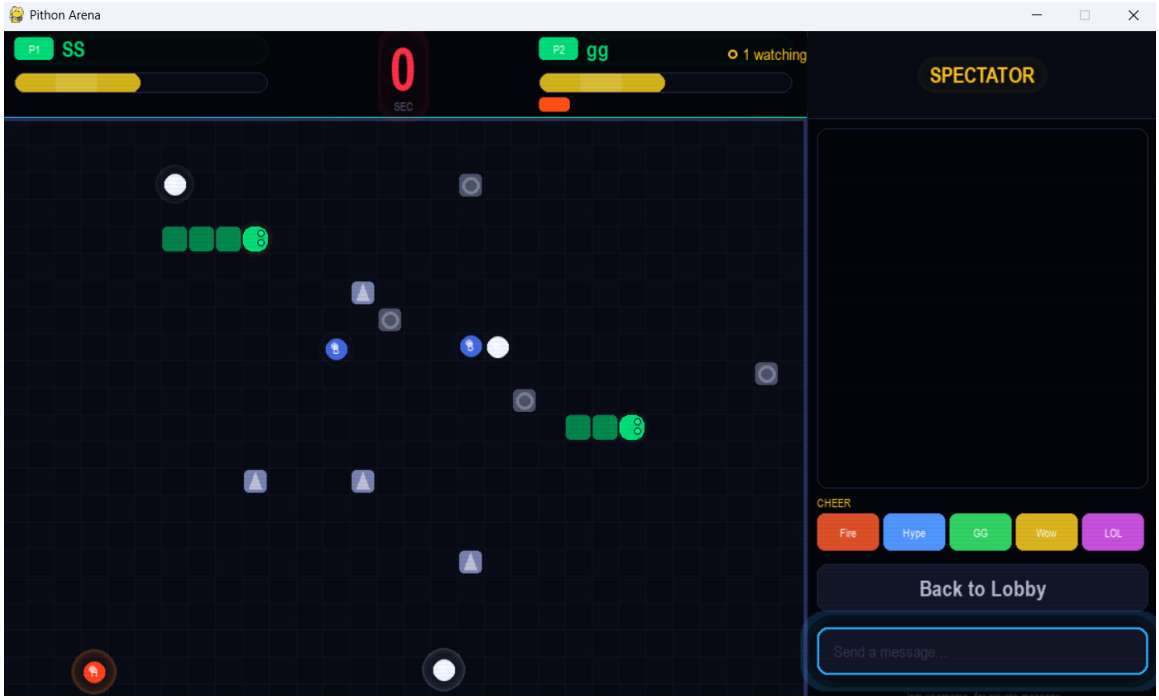
Connection & Login UI



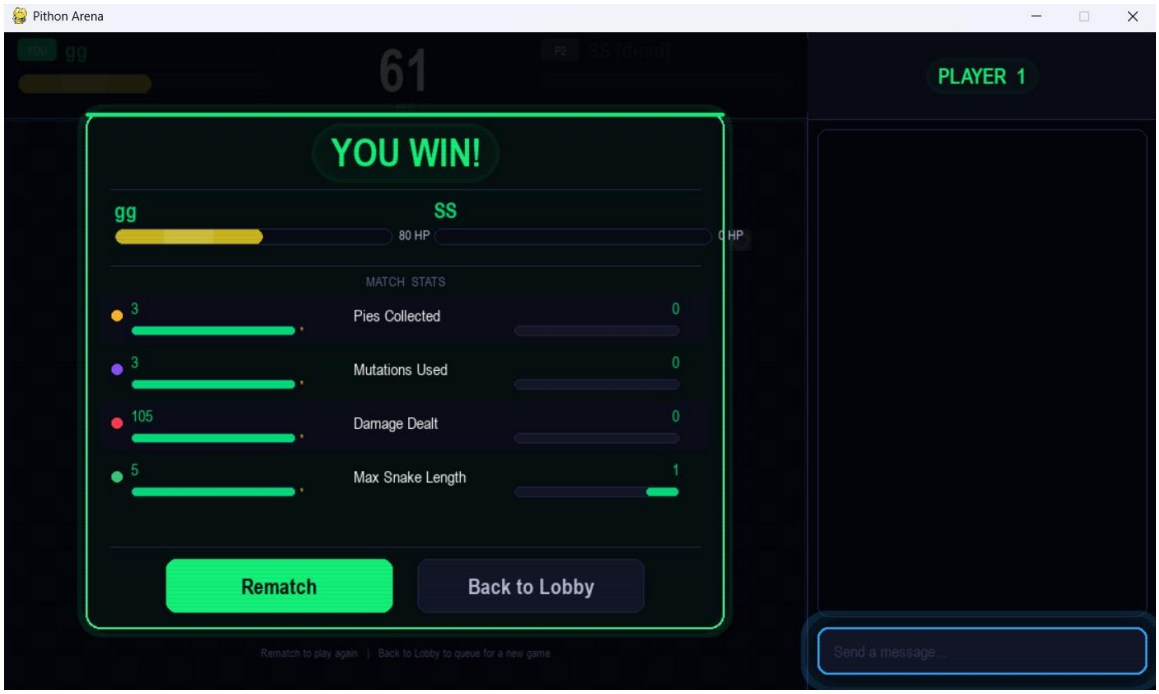
The matchmaking Lobby interface



1v1 Combat



User spectating the match



Match Summary & Statistics

Appendix B: Project Task Breakdown

Table 3: Distribution of responsibilities among team members.

Responsibility Description	Responsible Implementation Lead
Multithreading	Sari Sfeir
TCP Socket logic	Sari Sfeir
protocol.py	Alwalid Baydoun
game.py	Alwalid Baydoun
Pygame	Sari Sfeir/ Alwalid Baydoun / Georgio Sleiman
UI and animations	Sari Sfeir/ Alwalid Baydoun / Georgio Sleiman